

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
CENTRO UNIVERSITARIO GUASTATOYA
FACULTAD: INGENIERÍA EN SISTEMAS
CURSO: ANÁLISIS DE SISTEMAS II
CÁTEDRA: _____
SECCIÓN: A



APLICACIÓN DEL PRINCIPIO ISP AL DISEÑO DEL DASHBOARD DE OCUPACIÓN HOSPITALARIA

Consolidación y consulta de ocupación por sala y estado de cama

URL DEL REPOSITORIO:
<https://github.com/compilations-teams/sistema-hospitalario-integrado-SistemasII-2026>
RAMA: feature/asii-09-dashboard-de-ocupacion-hospitalaria-javifa2324
ETIQUETA EVALUADA: asii-09-semana2

NOMBRE DEL ESTUDIANTE: Javier Alexander Fajardo
López
NÚMERO DE CARNÉ: _____
FECHA DE ENTREGA: 14 de agosto de 2026

ÍNDICE

1. Introducción	3
2. Módulo, flujo y alcance	3
3. Requisitos funcionales y no funcionales	4
4. Criterios de aceptación	4
5. Aplicación del principio ISP	5
6. Diseño antes de aplicar ISP	5
7. Diseño después de aplicar ISP	6
8. Responsabilidades y dependencias	7
9. Evidencia y validación	8
10. Conclusión	8
11. Bibliografía	8

1. Introducción

En este trabajo aplico el principio de Segregación de Interfaces (ISP) al diseño del Dashboard de ocupación hospitalaria. El flujo que analizo es la consolidación y consulta de ocupación por sala y estado de cama. La idea no es cambiar lo que hace el módulo, sino revisar cómo están organizadas sus dependencias para evitar que un componente tenga que conocer operaciones que realmente no utiliza.

El ejercicio parte de un diseño sencillo donde varias consultas del dashboard están reunidas en una sola interfaz. Después se separan esas operaciones en interfaces más pequeñas según la necesidad de cada parte del dashboard. Con esto se busca que el diseño quede más claro, fácil de mantener y menos sensible a cambios que no le corresponden.

2. Módulo, flujo y alcance

2.1 Módulo asignado

Mi módulo es el Dashboard de ocupación hospitalaria. Su función es mostrar de forma resumida cómo se encuentran las camas del hospital, permitiendo revisar la ocupación general, consultar por sala, filtrar por estado y ver indicadores relacionados con la ocupación.

2.2 Flujo trabajado

El flujo trabajado es la consolidación y consulta de ocupación por sala y estado de cama. Un usuario autorizado abre el dashboard, consulta el resumen, puede aplicar filtros y recibe la información disponible sin modificar directamente el estado de las camas.

2.3 Alcance

El alcance se limita a consultar y presentar información de ocupación. No incluye crear salas, registrar camas, admitir pacientes, trasladarlos, dar altas ni liberar camas. Esas acciones pertenecen a otros procesos y no son necesarias para demostrar ISP en este módulo.

2.4 Objetivo del cambio

El objetivo es separar las dependencias del dashboard para que cada cliente utilice una interfaz pequeña y enfocada. El resumen no necesita depender de los filtros, los filtros no necesitan depender del detalle de una cama y el componente de métricas no necesita conocer operaciones que no usa.

3. Requisitos funcionales y no funcionales

3.1 Requisitos funcionales (RF)

ID	Requisito funcional
RF-01	Consultar el resumen general de ocupación, mostrando al menos total de camas, ocupadas y disponibles.
RF-02	Consultar la ocupación de una sala específica.
RF-03	Filtrar la información por estado de cama.
RF-04	Obtener el porcentaje o indicador de ocupación a partir de los datos consolidados.
RF-05	Consultar el detalle básico de una cama cuando el usuario lo solicite.
RF-06	Mostrar un estado vacío cuando no existan datos y un mensaje claro cuando ocurra un error de consulta.
RF-07	Permitir la consulta únicamente a usuarios con acceso autorizado al dashboard.

3.2 Requisitos no funcionales (RNF)

ID	Tipo	Requisito no funcional
RNF-01	Seguridad	La consulta debe respetar la autenticación y los permisos definidos para el dashboard.
RNF-02	Rendimiento	Con datos de prueba normales, la consulta principal debería responder en un tiempo que no interrumpa el uso del dashboard; para esta actividad se propone un máximo de 3 segundos.
RNF-03	Mantenibilidad	Los componentes deben depender solo de las operaciones que realmente utilizan, evitando interfaces demasiado grandes.
RNF-04	Usabilidad	Los estados de carga, vacío y error deben ser fáciles de entender para el usuario.
RNF-05	Trazabilidad	Los requisitos, criterios de aceptación y elementos del diseño deben mantener relación entre sí.

4. Criterios de aceptación

ID	Criterio de aceptación
CA-01	Si un usuario autorizado abre el dashboard y existen datos, debe visualizar el resumen general de ocupación.
CA-02	Si el usuario selecciona una sala, el resultado debe mostrar únicamente la información correspondiente a esa sala.
CA-03	Si el usuario selecciona un estado de cama, el resultado debe limitarse a las camas que cumplan ese estado.
CA-04	El porcentaje de ocupación debe ser coherente con los valores de total y camas ocupadas usados en la misma respuesta.
CA-05	Si no existen registros para la consulta, el sistema debe mostrar un estado vacío y no un error genérico.
CA-06	Si ocurre un fallo durante la consulta, el usuario debe recibir un mensaje de error comprensible.
CA-07	Si el usuario no tiene autorización, la consulta debe ser rechazada.
CA-08	El componente de resumen debe depender solo de la interfaz de resumen; los filtros, métricas y detalle deben depender de sus interfaces correspondientes.

5. Aplicación del principio ISP

La fuente indicada para la actividad explica que, al aplicar Interface Segregation, es preferible trabajar con varias interfaces pequeñas que con pocas interfaces cargadas de métodos. La razón es sencilla: una clase o un cliente no debería quedar obligado a implementar o depender de operaciones que no necesita.

“ninguna clase debe de estar obligada a implementar métodos que no necesita” (MVP Cluster, “Principios básicos del diseño de software”).

En mi módulo ese problema aparece si todo el dashboard depende de una sola interfaz que mezcla resumen, filtros, métricas y detalle de cama. Aunque todas esas operaciones pertenezcan al mismo módulo, no todos los clientes las utilizan. Por eso la mejora consiste en separar la interfaz de acuerdo con las necesidades reales de cada consumidor.

6. Diseño antes de aplicar ISP

En el diseño inicial se utiliza una sola interfaz llamada `OccupancyDashboardInterface`. Esta interfaz contiene todas las operaciones principales del dashboard. `OccupancyService` la implementa y los distintos componentes del dashboard dependen de ella.

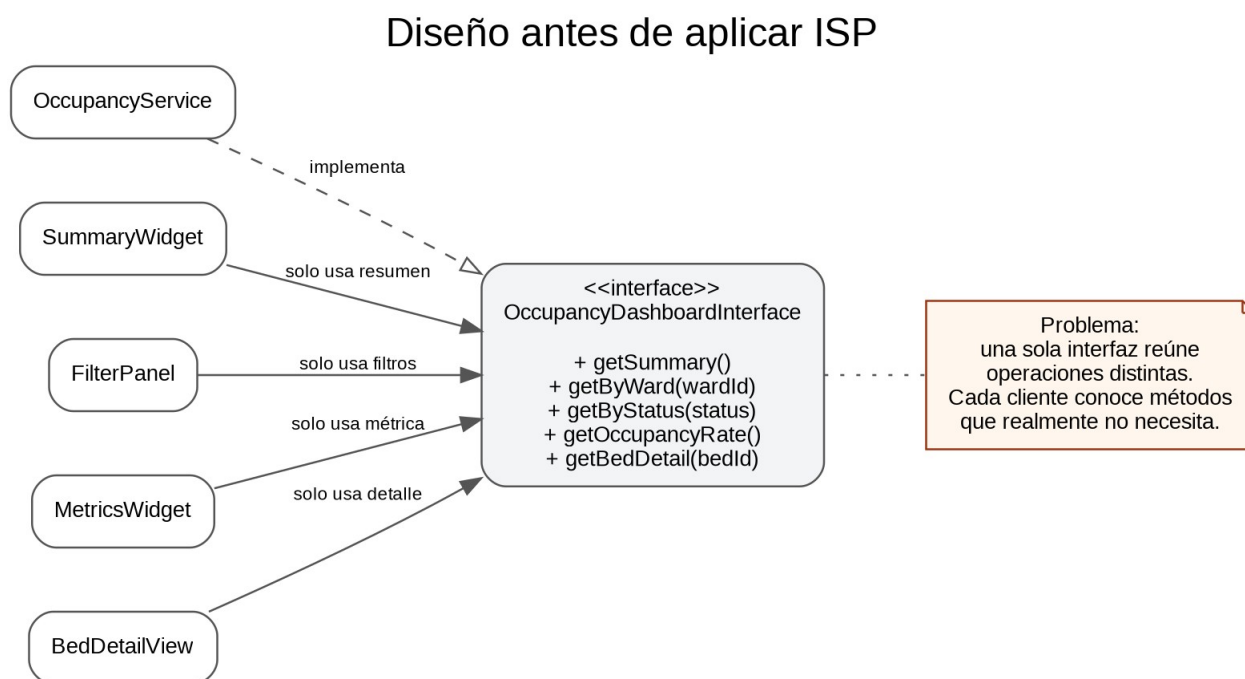


Figura 1. Diseño inicial: una sola interfaz concentra operaciones que no todos los clientes necesitan.

El problema no es que los métodos estén relacionados con ocupación, sino que la interfaz es más grande de lo que necesita cada cliente. `SummaryWidget` solo necesita `getSummary()`, `MetricsWidget` solo necesita `getOccupancyRate()` y `BedDetailView` solo necesita `getBedDetail()`. Sin embargo, todos quedan acoplados a la interfaz completa.

7. Diseño después de aplicar ISP

Después de aplicar ISP, la interfaz grande se separa en cuatro interfaces pequeñas. No es necesario crear un servicio distinto para cada una: OccupancyService puede implementar varias interfaces, mientras cada cliente depende únicamente de la que necesita.

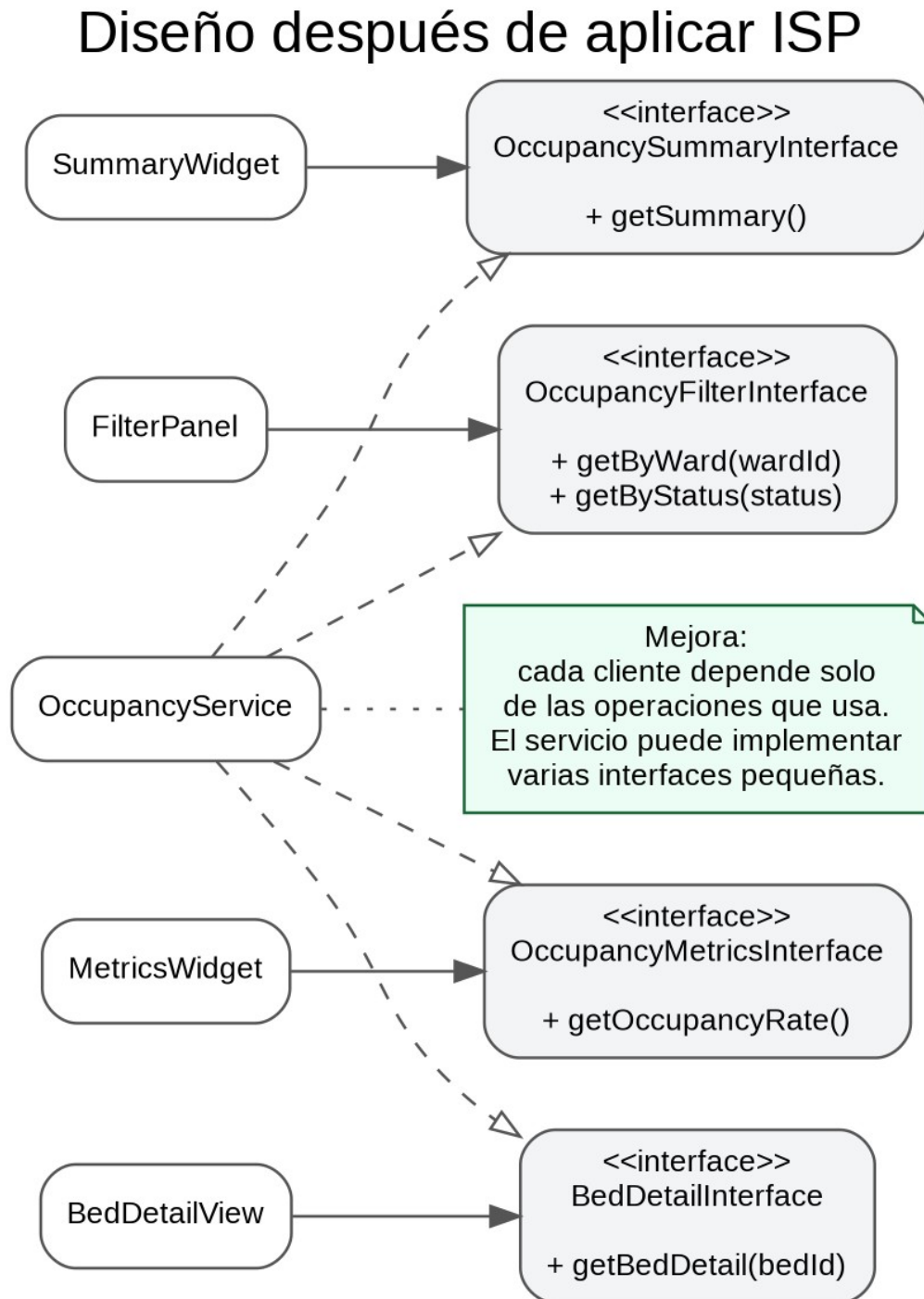


Figura 2. Diseño después de ISP: cada cliente depende de una interfaz enfocada.

8. Responsabilidades y dependencias

Elemento	Responsabilidad	Quién lo usa
OccupancySummaryInterface	Entregar el resumen general de ocupación.	SummaryWidget
OccupancyFilterInterface	Consultar por sala o por estado de cama.	FilterPanel
OccupancyMetricsInterface	Entregar el indicador o porcentaje de ocupación.	MetricsWidget
BedDetailInterface	Consultar el detalle básico de una cama.	BedDetailView
OccupancyService	Resolver las consultas y ofrecer las capacidades definidas por las interfaces.	Implementa las interfaces necesarias

La mejora principal es que un cambio en filtros no obliga al componente de resumen a depender de esa modificación. De la misma forma, agregar o ajustar una métrica queda separado del detalle de cama. Esto reduce dependencias innecesarias sin hacer el diseño más complicado de lo necesario.

8.1 Comparación antes y después

Aspecto	Antes	Después
Interfaces	Una interfaz grande para todo el dashboard.	Varias interfaces pequeñas por necesidad.
Dependencia de clientes	Cada cliente conoce métodos que no usa.	Cada cliente conoce solo lo que necesita.
Impacto de cambios	Un cambio en la interfaz puede alcanzar a varios clientes.	El cambio queda más acotado a la interfaz relacionada.
Mantenimiento	Más acoplamiento entre funciones diferentes.	Dependencias más claras y fáciles de revisar.

9. Evidencia y validación

La entrega incluye las fuentes editables isp-antes.puml e isp-despues.puml, además de las imágenes de ambos diseños. Los nombres de interfaces y métodos usados en los diagramas coinciden con la explicación del documento y con los criterios de aceptación relacionados con ISP.

Para la evidencia Git se debe usar la salida real del repositorio después de copiar estos archivos al worktree y realizar el commit. No se colocan hashes inventados. El archivo EVIDENCIA_GIT.md contiene los comandos para obtener la rama, el historial, el árbol de archivos, la URL remota y el commit final.

9.1 Validaciones realizadas

- El diseño inicial muestra una interfaz amplia de la que dependen clientes con necesidades distintas.
- El diseño final separa resumen, filtros, métricas y detalle de cama.
- Cada cliente del diseño final depende de una sola interfaz acorde con lo que utiliza.
- Los RF, RNF y criterios de aceptación se relacionan con el mismo flujo del dashboard.
- No se utilizaron datos clínicos reales ni información identificable de pacientes.
- Las fuentes .puml quedan disponibles para modificar un elemento durante la defensa oral.

10. Conclusión

Con esta actividad logré aplicar ISP al diseño del Dashboard de ocupación hospitalaria sin cambiar el objetivo del módulo. El punto más importante fue identificar que una sola interfaz para resumen, filtros, métricas y detalle hacía que los clientes dependieran de operaciones que no usaban. Al separar esas capacidades, cada parte del dashboard queda conectada únicamente con lo que necesita.

La limitación de este ejercicio es que se trabaja a nivel de diseño y no incluye una implementación completa del servicio. Aun así, los requisitos, criterios de aceptación, diagramas editables y comparación antes/después permiten comprobar de forma clara dónde estaba el problema y cómo se corrige aplicando ISP.

11. Bibliografía

MVP Cluster. “Principios básicos del diseño de software (SOLID)”. Disponible en:

<https://mvpcluster.com/disenio-de-software-2/>. Consulta indicada en la guía de la actividad: 31 de julio de 2026.

PlantUML. Documentación y proyecto oficial para diagramas como código. <https://plantuml.com/>.

Nota para la entrega: la URL y la rama de la portada corresponden al repositorio de trabajo. La etiqueta asii-09-semana2 debe crearse sobre el commit final de esta actividad y publicarse en origin.